API Security Risk Analysis Report

Internship Program: Future Interns – Cyber Security

Intern Name: Arra Vaishnavi Reddy

Task Code: FUTURE_CS_03

1. Objective

The objective of this task is to analyze a public API endpoint and identify potential security risks related to authentication, authorization, data exposure, and access control. This assessment helps in understanding common API security misconfigurations observed in real-world applications.

2. Tool Used

• Postman – Used to send HTTP requests and analyze API responses.

3. API Tested

• Endpoint: https://jsonplaceholder.typicode.com/users

• HTTP Method: GET

• Authentication: Not implemented (Public API)

4. Methodology / Steps Performed

1. Opened the Postman application.
2. Created a new HTTP GET request.
3. Entered the public API endpoint.
4. Sent the request without authentication credentials.
5. Reviewed the API response headers and body.

5. Observations

1. The API returned a successful 200 OK response.
2. User-like data such as name, email, address, and company details were visible.
3. No authentication, authorization, or access restrictions were enforced.

6. Security Issues Identified

6.1 Missing Authentication

The API allows access to user-related data without requiring any authentication mechanism. In real-world scenarios, this could lead to unauthorized data access and privacy violations.

6.2 Excessive Data Exposure

The API response exposes unnecessary user-related information such as email addresses and physical address details. This increases the risk of data misuse and privacy breaches.

6.3 Lack of Access Control and Rate Limiting

The API does not enforce role-based access control or request rate limiting, allowing unlimited requests. This could result in data scraping or denial-of-service risks.

7. Risk Severity Assessment

Risk Level: Medium. Although the API provides mock data for testing purposes, the identified issues would be considered serious vulnerabilities if present in a production environment.

8. Recommendations

1. Implement strong authentication mechanisms such as API keys, OAuth 2.0, or JWT.
2. Enforce role-based access control (RBAC).
3. Apply rate limiting and request throttling.
4. Restrict API responses to only required data fields.
5. Enable logging and monitoring to detect suspicious activities.

9. Conclusion

This assessment demonstrates the importance of securing APIs against unauthorized access and excessive data exposure. While the tested API is intended for public testing, similar configurations in real-world applications can lead to significant security and compliance risks. Proper authentication, authorization, and monitoring are essential for robust API security.